Claude Code

Complete Architecture Deep Analysis

Comprehensive End-to-End System Design Reference

Agentic Loop • Tools • Sub-Agents • Permissions • Context Engineering

Version 1.0 — April 2, 2026

Based on analysis of claw-code-main repository

Clean-room architectural study

Table of Contents

Part I — Foundation

1. Executive Summary & Key Metrics 4
2. Repository Structure & Codebase Overview 5

Part II — Core Agent Architecture

3. The ReAct Agentic Loop 7
4. Tool System & Registry 10
5. Permission & Safety Model 14
6. Hook Middleware Pipeline 17

Part III — Multi-Agent & Extensibility

7. Sub-Agent Delegation Architecture 19
8. Skills System 23
9. Plugin Architecture 25
10. MCP Integration 28

Part IV — Context Engineering

11. Context Management & Compaction 30
12. System Prompt Assembly 33
13. Memory System 36
14. Session Persistence & Token Tracking 37

Part V — Deployment & Integration

15. Server Mode & HTTP API 39
16. Multi-Provider API Layer 41

17. IDE Bridge & Terminal UI 43

Part VI — Analysis

18. Comparative Analysis & Key Takeaways 45

Part I — Foundation

1. Executive Summary

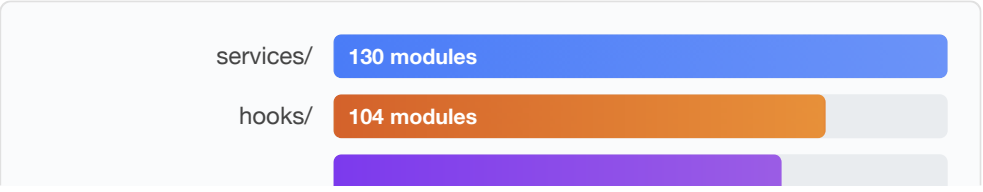
> *Claude Code is Anthropic's official agentic CLI for Claude — a ReAct-loop coding assistant that operates across CLI, IDE extension, and HTTP server deployment modes. This document presents a complete architectural analysis based on clean-room study of the claw-code-main repository.*

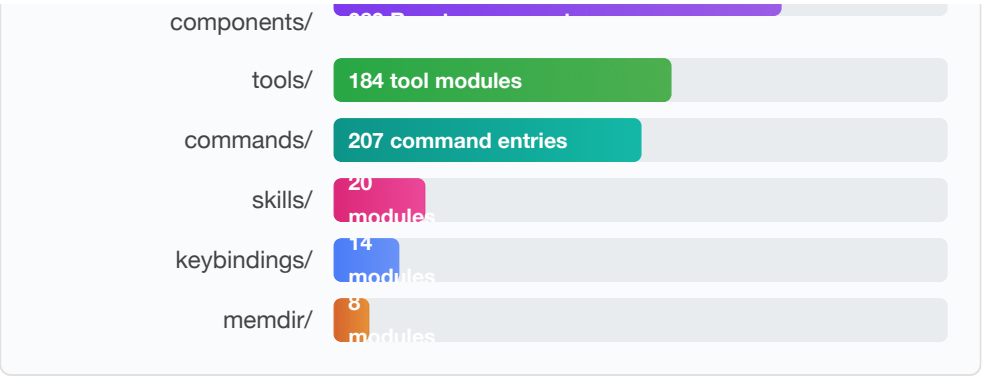
1,902 TYPESCRIPT FILES	**184** TOTAL TOOLS	**207** SLASH COMMANDS
35 SUBSYSTEMS	**5** SUB-AGENT TYPES	**8** RUST CRATES

Core Design Principles

1. **Safety-first tool execution** — Permission gates + hook middleware + sandbox isolation form three layers of defense
2. **Streaming-first design** — SSE parsing, real-time markdown rendering, incremental tool output for responsive UX
3. **Context engineering** — Automatic hierarchical compaction, instruction file discovery, token budgeting, and prompt caching

Subsystem Module Distribution




components/
tools/
184 tool modules
commands/
207 command entries
skills/
20 modules
keybindings/
14 modules
memdir/
8 modules

2. Repository Structure

Dual-Language Architecture

The repository contains two parallel implementations: the original TypeScript source (archived as reference data) and an active Rust port. A Python compatibility layer provides analysis tooling.

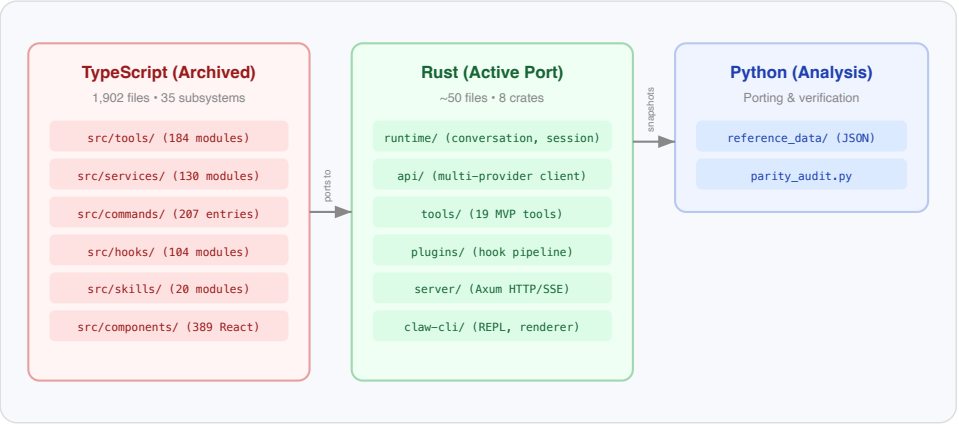


Figure 2.1 — Dual-language repository architecture with Python analysis bridge

Rust Crate Dependency Graph

Crate	Depends On	Purpose
`claw-cli`	api, runtime, tools, commands, plugins, compat-harness	Interactive CLI binary & REPL
`tools`	api, runtime, plugins	Tool registry, schemas, execution
`commands`	runtime, tools	Slash command handling
`server`	runtime	HTTP/SSE API server (Axum)
`plugins`	(standalone)	Plugin model, hooks, lifecycle
`runtime`	lsp	Session, compaction, MCP, config
`api`	(standalone)	Multi-provider API client

`lsp`	(standalone)	Language Server Protocol client

Part II — Core Agent Architecture

3. The ReAct Agentic Loop

> *The heart of Claude Code is a Reason+Act loop where the LLM alternates between generating text/tool calls and processing tool results until the task is complete.*

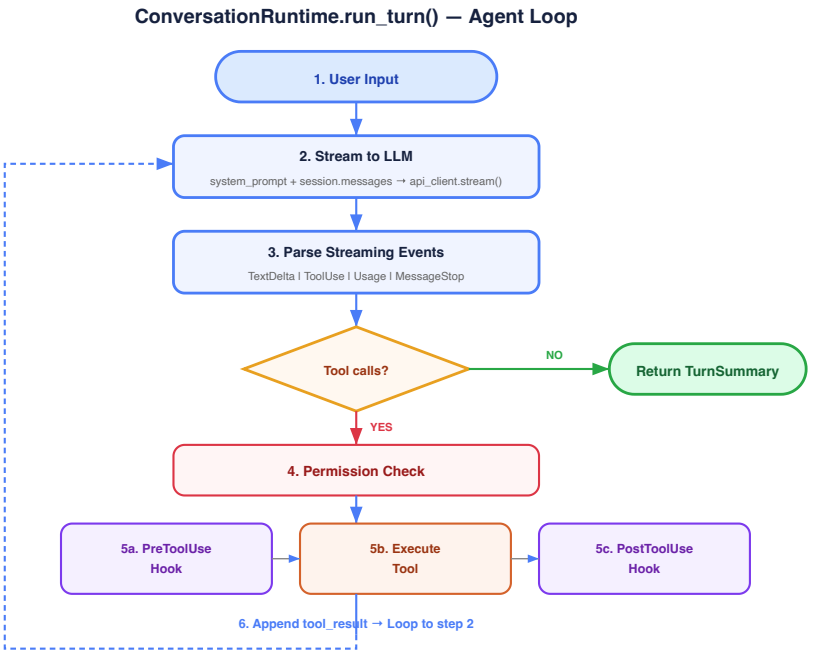


Figure 3.1 — The ReAct agent loop: stream response, extract tools, execute with permission + hooks, loop

Runtime Data Model

```
pub struct ConversationRuntime<C, T> {
    session: Session,              // Vec<ConversationMessage>
    api_client: C,                 // impl ApiClient (trait)
    tool_executor: T,              // impl ToolExecutor (trait)
    permission_policy: PermissionPolicy,
    system_prompt: Vec<String>,
    max_iterations: usize,         // usize::MAX for main, 32 for sub-agen
```

```
    usage_tracker: UsageTracker,
    hook_runner: HookRunner,
}
```

Key Insight: Trait-Based Abstractions

Both `ApiClient` and `ToolExecutor` are traits, enabling testing with mock implementations and supporting different execution models (main agent vs sub-agent tool whitelisting).

Message Types

Role	Block Types	Purpose
System	Text	System instructions, compaction summaries
User	Text	User messages and queries
Assistant	Text + ToolUse	LLM responses (text interleaved with tool calls)
Tool	ToolResult	Tool execution results (output + is_error flag)

4. Tool System & Registry

> *The tool system provides 19 built-in tools (Rust MVP) out of 184 total tools in the full TypeScript inventory, organized through a GlobalToolRegistry that combines built-in and plugin-provided tools.*

Built-in Tool Inventory

Tool	Permission	Category	Description
`bash`	DangerFull	Execution	Shell commands + timeout + sandbox
`read_file`	ReadOnly	File I/O	Read with offset/limit
`write_file`	WorkspaceWrite	File I/O	Create/overwrite files
`edit_file`	WorkspaceWrite	File I/O	String replacement (old → new)
`glob_search`	ReadOnly	Search	Find files by glob pattern
`grep_search`	ReadOnly	Search	Regex content search (ripgrep)
`WebFetch`	ReadOnly	Web	Fetch URL + summarize
`WebSearch`	ReadOnly	Web	Web search with domain filters
`Agent`	DangerFull	Delegation	Spawn sub-agents
`Skill`	ReadOnly	Skills	Load skill definitions

`TodoWrite`	WorkspaceWrite	Task Mgmt	Structured task lists
`ToolSearch`	ReadOnly	Discovery	Search deferred tools
`NotebookEdit`	WorkspaceWrite	Notebook	Jupyter cell editing
`SendUserMessage`	ReadOnly	Communication	Messages + attachments to user
`Config`	WorkspaceWrite	Settings	Get/set configuration
`Sleep`	ReadOnly	Utility	Non-blocking delay
`StructuredOutput`	ReadOnly	Output	Return structured JSON
`REPL`	DangerFull	Execution	Language-specific code exec
`PowerShell`	DangerFull	Execution	Windows PowerShell

Tool Permission Distribution

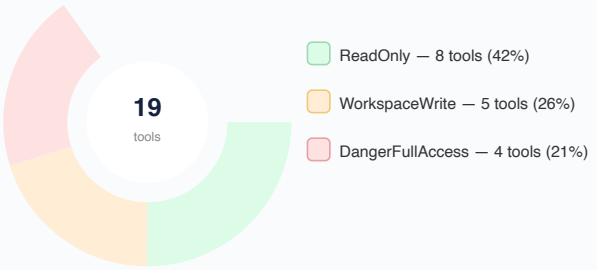


Figure 4.1 — Tool permission level distribution (Rust MVP)

5. Permission & Safety Model

> *A three-tier permission hierarchy with prompt escalation, sandbox isolation for bash, and hook middleware provides defense-in-depth for AI tool execution.*

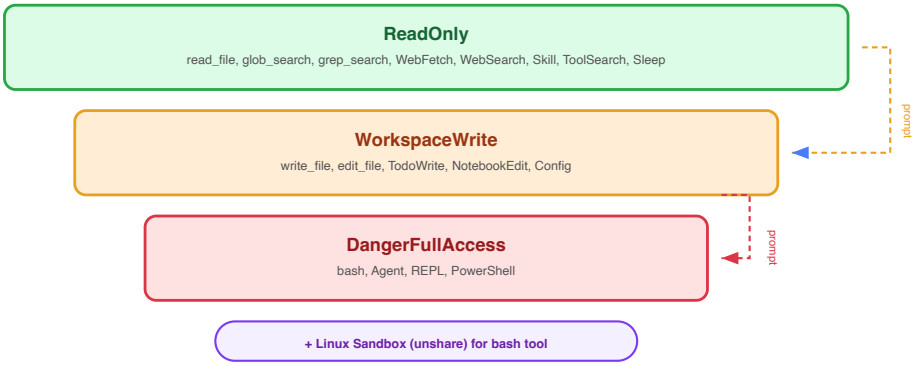


Figure 5.1 — Three-tier permission hierarchy with escalation prompts

Authorization Decision Flow

Current Mode	Tool Requires	Outcome
DangerFullAccess	Any	Auto-Allow
WorkspaceWrite	ReadOnly / WorkspaceWrite	Auto-Allow
WorkspaceWrite	DangerFullAccess	Prompt User
ReadOnly	WorkspaceWrite+	Deny
Prompt	Any privileged	Prompt User
Allow	Any	Auto-Allow

6. Hook Middleware Pipeline

> *Pre/post tool execution hooks provide extensible safety, audit, and policy enforcement through shell commands with a simple exit code protocol.*

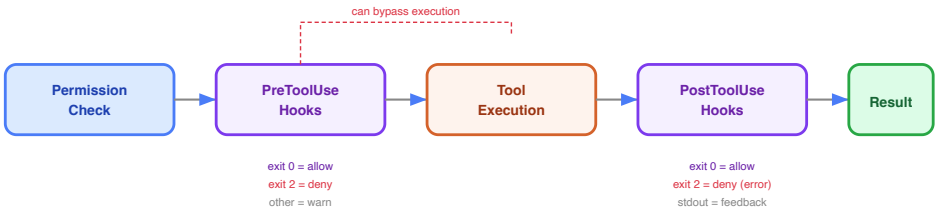


Figure 6.1 — Hook middleware pipeline with exit code protocol

Hook Environment Variables

Variable	Available In	Content
`HOOK_EVENT`	Both	"PreToolUse" or "PostToolUse"
`HOOK_TOOL_NAME`	Both	Tool name (e.g., "bash")
`HOOK_TOOL_INPUT`	Both	Raw input JSON string
`HOOK_TOOL_OUTPUT`	PostToolUse	Tool execution output
`HOOK_TOOL_IS_ERROR`	PostToolUse	"0" or "1"

Part III — Multi-Agent & Extensibility

7. Sub-Agent Delegation Architecture

> *Claude Code spawns specialized sub-agents in isolated OS threads with per-type tool whitelists, fresh conversation sessions, and persistent output for audit and recovery.*

Built-in Sub-Agent Types

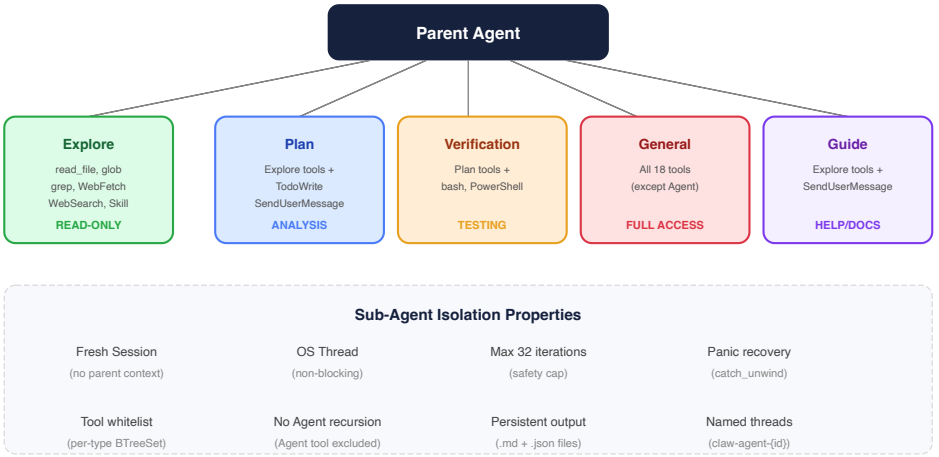


Figure 7.1 — Five built-in sub-agent types with tool access hierarchy and isolation properties

8. Skills System

Skills are markdown files with YAML frontmatter that inject instructions into the conversation when invoked through the `Skill` tool.

Bundled Skills (14 total)

Skill	Purpose
`batch`	Batch processing of multiple files
`clawApi`	Build apps with the Claude API
`debug`	Systematic debugging workflow
`keybindings`	Customize keyboard shortcuts
`loop`	Run prompts on recurring intervals
`remember`	Save information to memory
`scheduleRemoteAgents`	Cron-scheduled remote agents
`simplify`	Review code for quality and simplification
`skillify`	Create new skills from templates
`stuck`	Help when agent is stuck
`updateConfig`	Configure settings via hooks
`verify`	Verification before completion
`verifyContent`	Content verification logic
`loremIpsum`	Generate placeholder text

9. Plugin Architecture

Plugins extend Claude Code with custom tools, hooks, and lifecycle commands through a JSON manifest and subprocess execution model.

Plugin Execution Model

Subprocess Isolation

Plugin tools execute as separate processes with JSON input via stdin and environment variables (`CLAW_PLUGIN_ID` , `CLAW_TOOL_NAME` , `CLAW_TOOL_INPUT`). The plugin's root directory is used as CWD.

Plugin Types & Sources

Type	Location	Install	Update
Builtin	Compiled into binary	Automatic	Binary update
Bundled	Shipped with install	Auto-synced	Version update
External	`~/.claw/plugins/`	Manual (local/git)	Re-install

10. MCP (Model Context Protocol) Integration

MCP Transport Types

Transport	Config	Use Case
Stdio	`command` , `args` , `env`	Local subprocess servers
SSE	`url` , `headers` , `oauth`	Remote SSE endpoints
HTTP	`url` , `headers` , `oauth`	Remote HTTP endpoints
WebSocket	`url` , `headers`	Bidirectional streaming
SDK	`name`	Native integration
ManagedProxy	`url` , `id`	Claude infrastructure proxy

MCP Tool Naming Convention

`mcp__{normalized_server_name}__{normalized_tool_name}`

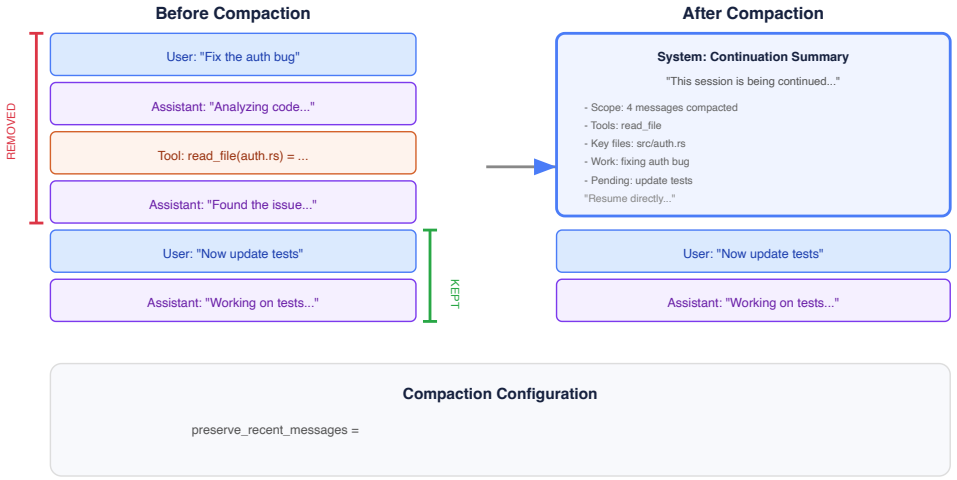
Example: Server "my-api" + tool "fetch" → `mcp__my_api__fetch`

Part IV — Context Engineering

11. Context Management & Compaction

> *Automatic hierarchical compaction preserves conversation context across arbitrary length sessions by summarizing older messages while keeping recent context verbatim.*

Compaction Algorithm



4 max_estimated_tokens = **10,000** Token estimation: characters / 4 + 1 per content block

Figure 11.1 — Compaction removes older messages and replaces them with a structured summary

12. System Prompt Assembly Pipeline

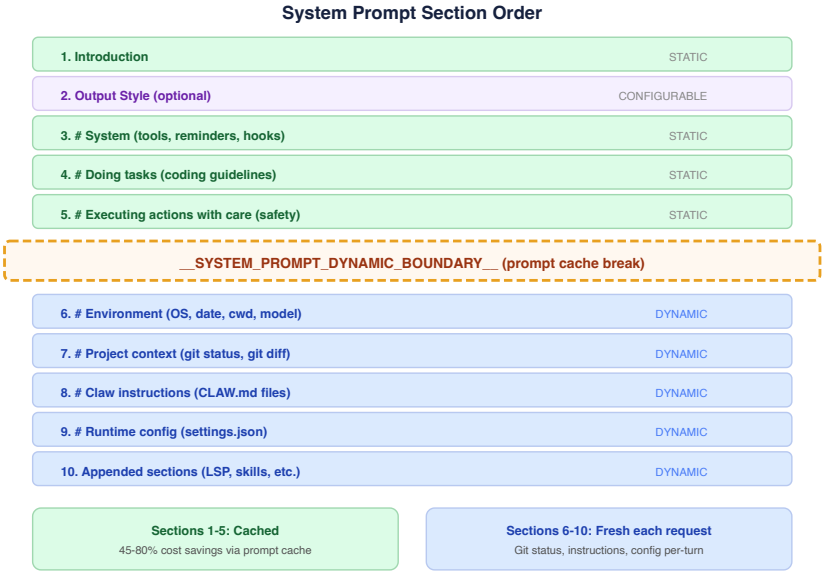


Figure 12.1 — System prompt assembly with dynamic boundary enabling prompt caching

Instruction File Discovery

The system walks the directory tree from root to CWD, checking 4 filenames at each level:

1. `CLAW.md` — Primary workspace instructions
2. `CLAW.local.md` — Local overrides (git-ignored)
3. `.claw/CLAW.md` — Hidden config instructions
4. `.claw/instructions.md` — Additional instructions

> **Instruction Budgets**
> Max 4,000 characters per file, 12,000 characters total. Content-hash deduplication prevents duplicate instructions from ancestor scopes.

13. Memory System (memdir)

Module	Purpose
`findRelevantMemories`	Semantic search across memory store
`memdir`	Core directory operations

`memoryAge`	Time-based decay and relevance
`memoryScan`	Periodic indexing of new entries
`memoryTypes`	Categories: conversation, project, user
`paths`	File system storage locations
`teamMemPaths`	Team-shared memory locations
`teamMemPrompts`	Team memory operation prompts

14. Session Persistence & Token Tracking

Token Usage Model

Model	Input $/1M	Output $/1M	Cache Write	Cache Read
Opus 4.6	$15.00	$75.00	$18.75	$1.50
Sonnet 4.6	$15.00	$75.00	$18.75	$1.50
Haiku 4.5	$1.00	$5.00	$1.25	$0.10

Part V — Deployment & Integration

15. Server Mode & HTTP API

API Endpoints

Endpoint	Method	Purpose	Response
`/sessions`	POST	Create session	`{ session_id }`
`/sessions`	GET	List sessions	`[ { id, created_at, message_count } ]`
`/sessions/{id}`	GET	Session details	Full conversation
`/sessions/{id}/message`	POST	Send message	204 No Content
`/sessions/{id}/events`	GET	SSE stream	Snapshot + incremental events

16. Multi-Provider API Layer

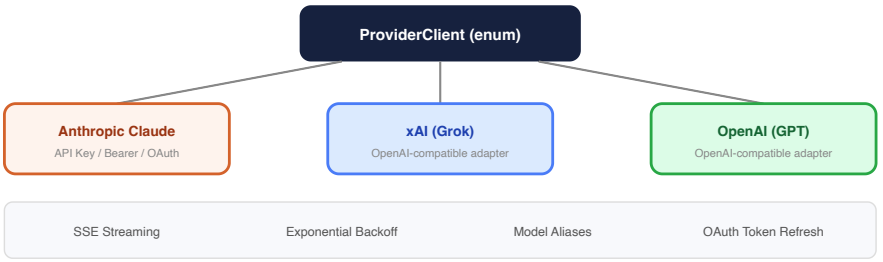


Figure 16.1 — Multi-provider API abstraction with unified streaming interface

17. IDE Bridge & Terminal UI

Terminal Rendering Stack

Library	Version	Purpose
crossterm	0.28	Terminal I/O, raw mode, events
pulldown-cmark	0.13	Markdown parsing to events
syntect	5	Syntax highlighting (base16-ocean.dark)
rustyline	15	Line editing with history + vim mode

LSP Integration Capabilities

- Document lifecycle management (open, change, save, close)
- Go-to-definition for symbol navigation
- Find references for usage tracking
- Workspace-wide diagnostics collection
- Context enrichment (combined diagnostics + definitions + references)

Part VI — Analysis

18. Comparative Analysis & Key Takeaways

Feature Comparison: Claude Code vs Our Agent

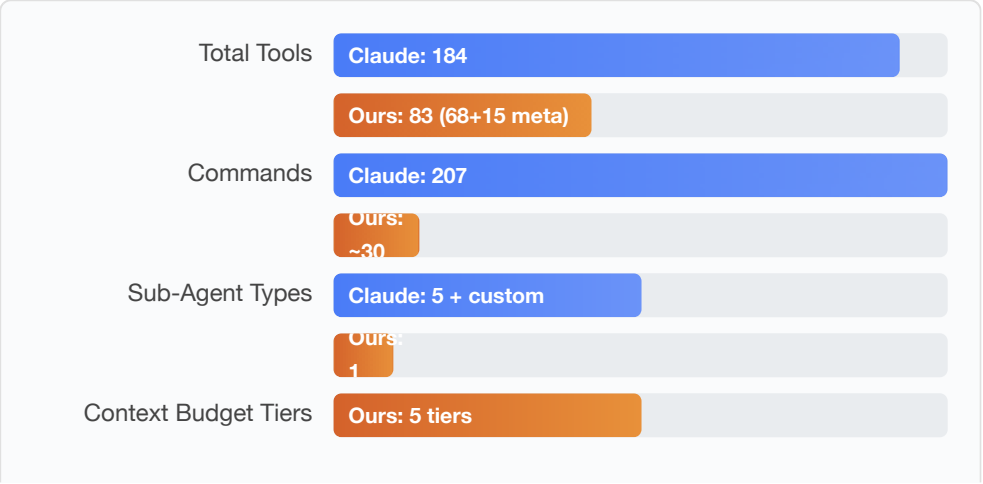
Claude Code Does Better

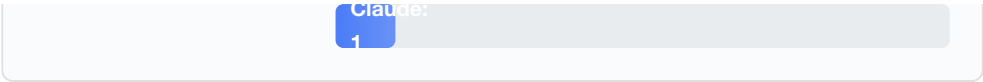
- Plugin ecosystem with manifest, tools, hooks, lifecycle
- MCP integration with 6 transport types + OAuth
- 5 typed sub-agents with tool whitelists
- Multi-provider support (Anthropic + xAI + OpenAI)
- Dynamic boundary for prompt caching
- Streaming boundary-aware markdown rendering

We Do Better

- IDE-native IntelliJ API access (vs LSP bridge)
- Meta-tool architecture (15 meta-tools, 138 actions)
- Multi-tier context budgets (5 levels vs 1)
- Event-sourced state (vs message-list append)
- Rich JCEF chat UI with React
- Cross-service integration (Jira, Bamboo, Sonar, etc.)

Key Metrics Comparison





Architectural Patterns to Adopt

Pattern	Claude Code	Our Action Item
Sub-agent types	Explore, Plan, Verification, Guide, General	Add Explore (read-only) and Verification (testing) types
Plugin architecture	manifest.json + subprocess tools	Consider for community extensions
Hook middleware	Pre/PostToolUse shell hooks	Add to settings.json config
Prompt caching	Dynamic boundary marker	Split system prompt static/dynamic
Tool search	Deferred tool loading	Reduce prompt size for large tool sets
Hierarchical compaction	Prior + new summary merge	Implement in condenser pipeline

Key Takeaway

Claude Code excels at extensibility (plugins, MCP, sub-agents) and cost optimization (prompt caching). We excel at IDE integration, context engineering depth, and cross-service workflows. The ideal architecture combines both approaches.

End of Document

Claude Code Architecture Deep Analysis
Version 1.0 — April 2, 2026
Based on clean-room analysis of claw-code-main repository

This document is for internal reference only.